

Lecture 3: Dependability

Syedhamidreza Mousavi

Outlines

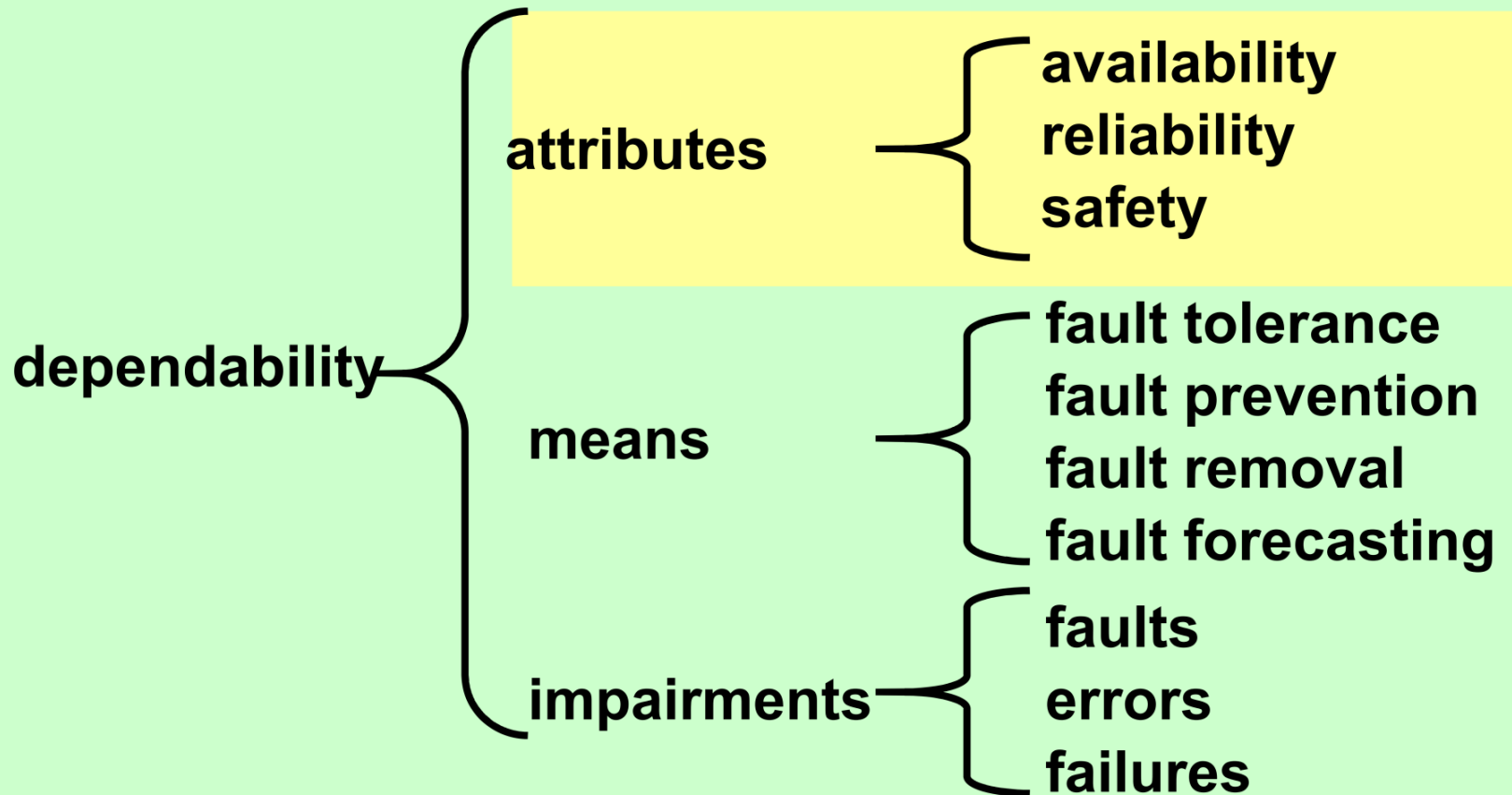
- 
- Dependability
 - Dependability tree
 - ❖ Dependability attributes
 - ❖ Dependability impairments
 - ❖ Dependability means

Dependability



Dependability is the ability of a system to deliver its intended level of service to its users

Dependability tree



Reliability

- $R(t)$ is the probability that a system operates without failure **in the interval $[0,t]$** , given that it worked at time 0
- We need high reliability when:
 - ❖ even momentary periods of incorrect performance are unacceptable (aircraft, heart pace maker)
 - ❖ no repair possible (satellite, spacecraft)

High reliability examples



- airplane:

- ❖ $R(\text{several hours}) = 0.999\ 999\ 9 = 0.9_7$

- spacecraft:

- ❖ $R(\text{several years}) = 0.95$

Reliability versus fault tolerance



- Fault tolerance is a technique that can improve reliability, but
 - ❖ a fault tolerant system **does not** necessarily have a high reliability
 - ❖ a system can be designed to tolerate any single error, but the probability of such error to occur can be so high that the reliability is very low

Reliability versus fault tolerance



- A highly reliable system is **not** necessarily fault tolerant
 - ❖ a very simple system can be designed using very good components such that the probability of hardware failing is very low
 - ❖ but if the hardware fails, the system cannot continue its functions

How fault tolerance helps



- Fault tolerance can improve a system's reliability by keeping the system operational when hardware or software faults occur
 - ❖ a computer system with one redundant processor can be designed to continue working correctly even if one of the processors fails

Availability



- $A(t)$ is the probability that a system is functioning correctly **at the instant of time t**
- depends on
 - ❖ how frequently the system becomes non-operational
 - ❖ how quickly it can be repaired

Steady-state availability



- Often the availability assumes a time-independent value after some initial time interval
- This value is called **steady-state** availability A_{ss}
- Steady-state availability is often specified in terms of **downtime** per year
 - ❖ $A_{ss} = 90\%$, downtime = 36.5 days/year
 - ❖ $A_{ss} = 99\%$, downtime = 3.65 days/year

Reliability versus availability



- reliability depends on an **interval** of time
- availability is taken at an **instant** of time
- a system can be highly available yet experience frequent periods of being non-operational as long as the length of each period is extremely short

High availability examples



■ examples

❖ transaction processing

- » ATM: $Ass=0.9_3$ (< 10 hours/year unavailable)
- » banking: $Ass=0.997$ (< 10 s/hour unavailable)

❖ computing

- » supercomputer centers
- » $Ass=0.997$ (< 10 days/year unavailable)

❖ embedded

- » telecom: $Ass=0.9_5$ (< 5 min/year unavailable)

How fault tolerance helps



- Fault tolerance can improve a system's availability by keeping the system operational when a failure occur
 - ❖ a spare processor can perform the functions of the system, keeping its available for use, while the primary processor is being repaired

Safety

- 
- Safety is the probability that a system will either perform its function correctly or will discontinue its operation in a safe way
 - System is safe
 - ❖ if it functions correctly, or
 - ❖ if it fails, it remains in a safe state

High safety examples



- nuclear energy
 - ❖ stop reactor if a problem occur
- banking
 - ❖ don't give the money if in doubt

Reliability versus safety



- Reliability is the probability that a system will perform its functions correctly
- Safety is the probability that a system will either work correctly or will stop in a manner that causes no harm

How fault tolerance helps



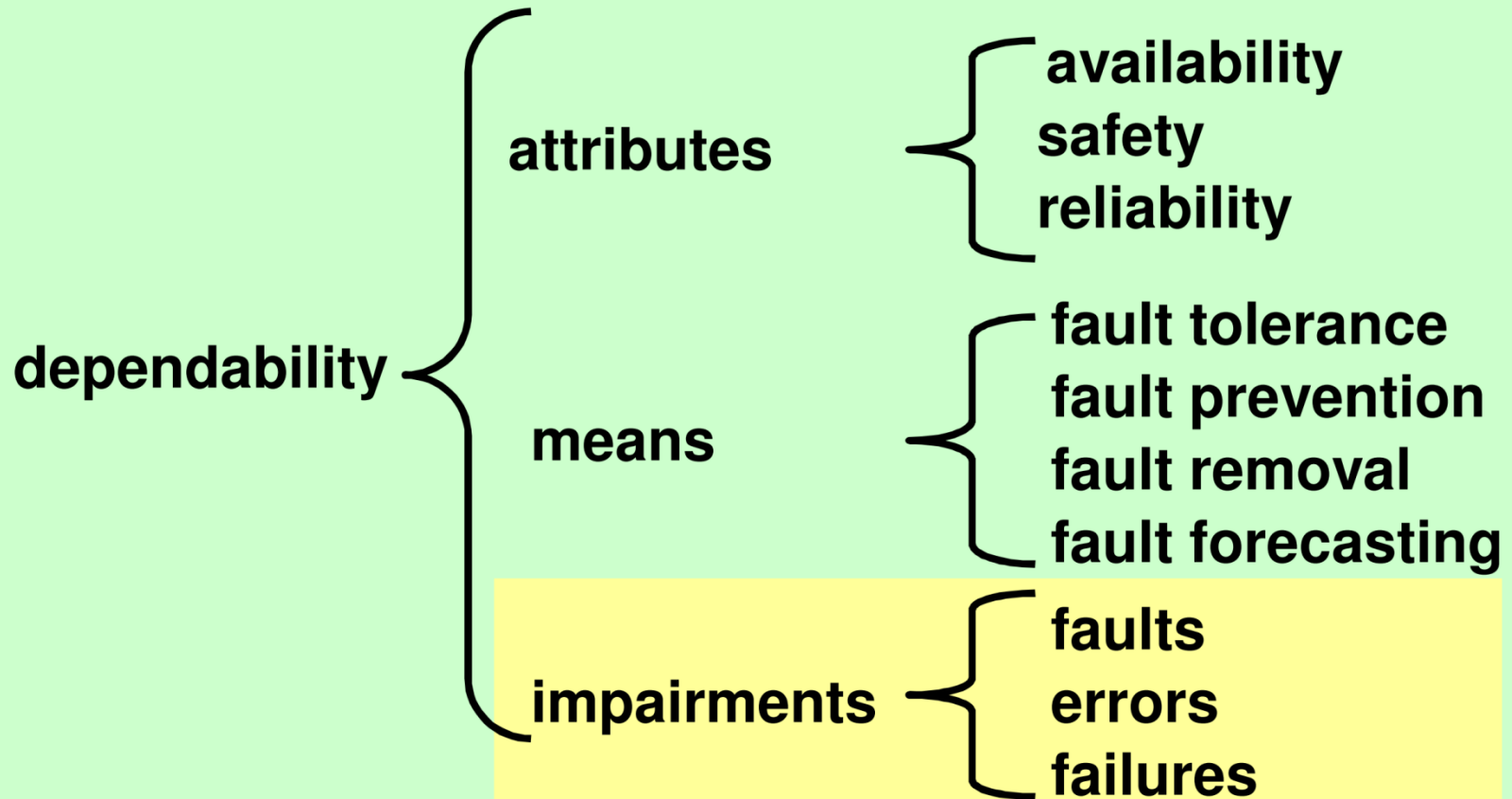
- Fault tolerance techniques can improve safety by turning a system off if a failure of a certain sort is detected
 - ❖ in a nuclear power plant the reaction process should be stopped if some discrepancy is detected

Summary: attributes of dependability



- reliability:
 - ❖ continuity of service
- availability:
 - ❖ readiness for usage
- safety:
 - ❖ non-occurrence of catastrophic consequences on environment

Dependability tree



Fault



Fault is a physical defect, imperfection or flaw that occurs in hardware or software

Example:

- short between wires
- break in transistor
- infinite program loop
- Single Event Transients (SET)

Error



Error is a deviation from correctness or
accuracy

Example: Suppose a line is physically shortened to 0 (there is a fault). As long as the value on line is supposed to be 0, there is no error.

Errors are usually associated with incorrect values in the system state.

Failure



Failure is a non-performance of some action
that is due or expected

Example: Suppose a circuit controls a lamp (0 = turn off, 1 = turn on) and the output is physically shortened to 0 (there is a fault). As long as the user wants the lamp off, there is no failure.

A system is said to have a failure if the service it delivers to the user deviates from compliance with the system specification.

Cause-and-effect relationship



- Faults can result in errors. Errors can lead to system failures
- Errors are the effect of faults. Failures are the effect of errors

Software

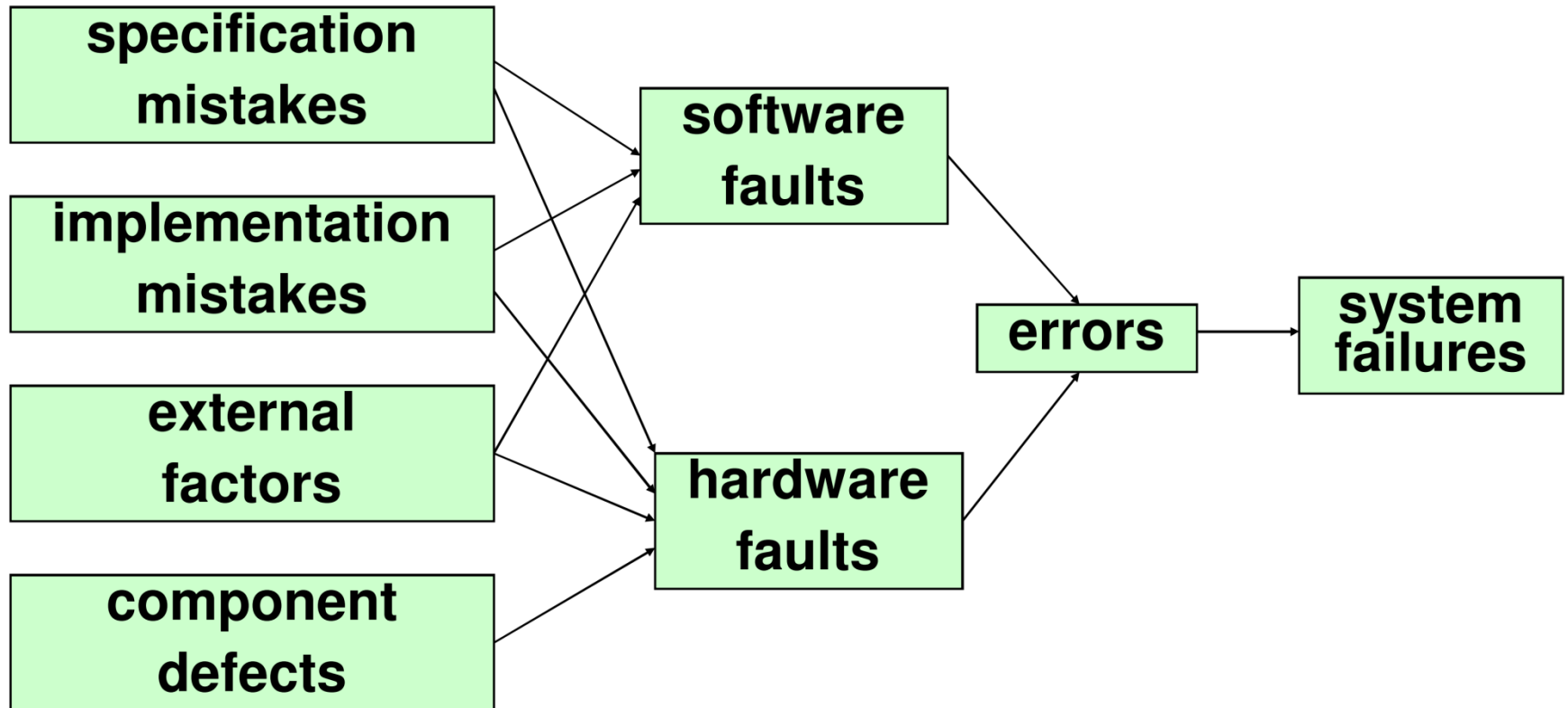
- 
- Definitions of physical, computational and system levels are more confusing when applied to software
 - ❖ physical level = program code
 - ❖ computational level = values of the program state
 - ❖ system level = software system running the program
 - Bug in a program is a fault. Possible incorrect values caused by this bug is an error. Possible crush of the operating system is a failure.

Origins of faults (HW/SW)



- specification mistakes
 - ❖ incorrect algorithms, incorrectly specified requirements (timing, power, environmental)
- implementation mistakes
 - ❖ poor design, software coding mistakes
- component defects
 - ❖ manufacturing imperfections, random device defects, components wear-outs
- external factors
 - ❖ radiation, lightning, operator mistakes

Cause-and-effect relationship



Hardware faults

- 
- **Fault duration** specifies the length of time that a fault is active
 - ❖ permanent fault
 - » remains in existence indefinitely if no corrective action is taken (stuck-at fault)
 - ❖ transient fault
 - » can appear and disappear within a very short period of time (caused by environmental, such as alpha particles, atmospheric neutrons, electrostatic discharge, electrical power drops, or overheating)
 - » 98 % of RAM faults
 - » DRAM: one single-bit transient fault per day per gigabyte of memory
 - ❖ intermittent fault
 - » appear, disappears and then reappears repeatedly (weak solder joint)

Fault models

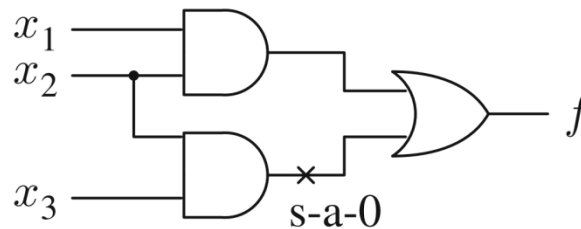
- 
- It is very difficult to analyze a system without assuming some fault models
 - ❖ hard to design test procedures
 - ❖ hard to simulate faults
 - To make the problem more manageable, we need to restrict our attention to a subset of all faults what can occur

Fault models

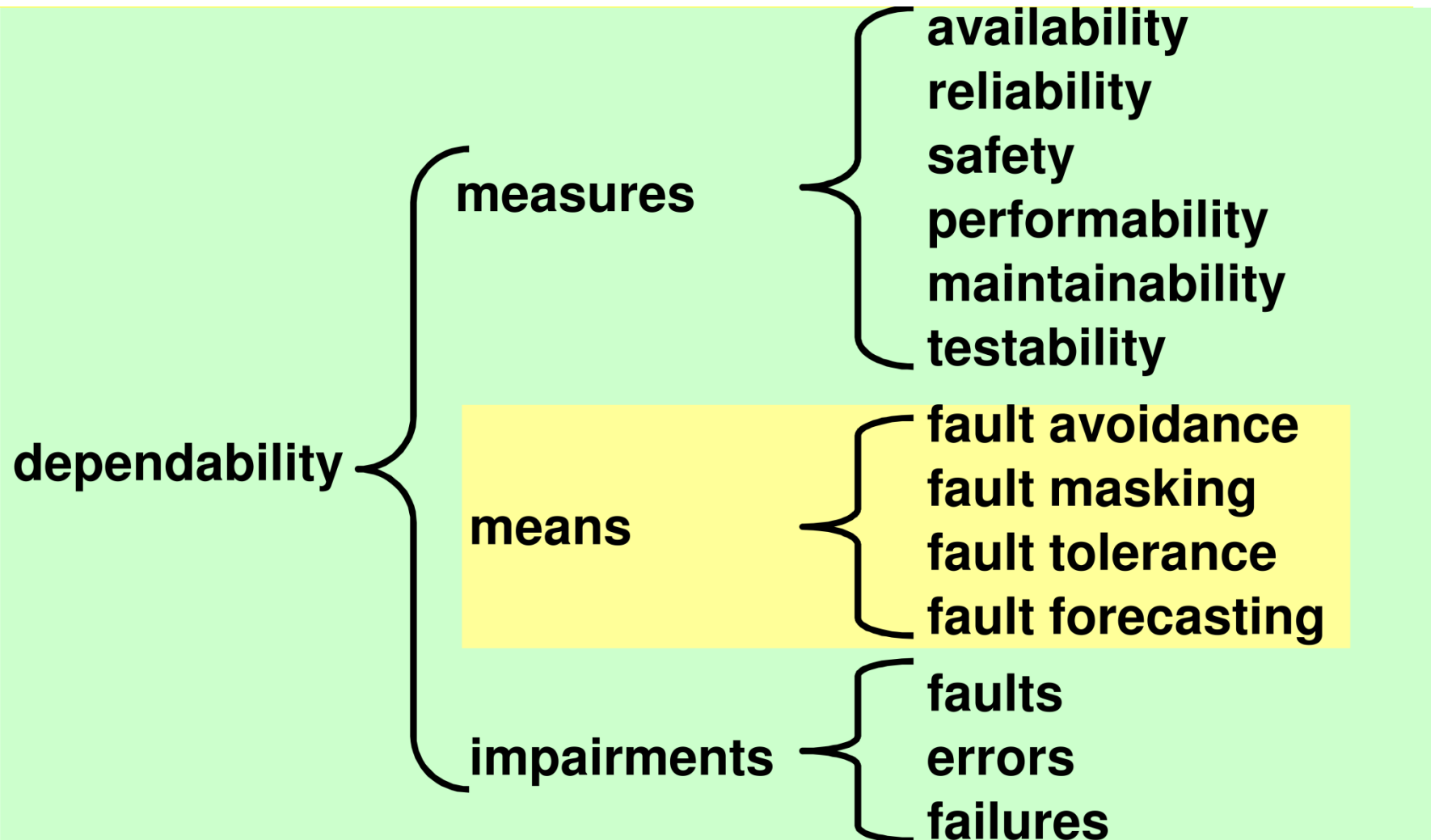
- 
- Fault model is a logical abstraction describing the **functional effect** of physical defect
 - ❖ Depending on the level of evaluation
 - Different levels of modeling
 - ❖ high, logic, transistor, layout
 - Different fault models
 - ❖ stuck-at

Logic stuck-at fault model

- most commonly used model
 - ❖ manufacturing defects
- the effect of the fault is modeled by having a line in the circuit permanently fixed to 0 or 1 value
- the basic functionality of the circuit is not changed
 - ❖ gates remain the same
 - ❖ combinational circuit is not transformed to sequential



Dependability tree



Dependability means



- Dependability means are techniques enabling the development of a dependable system:
 - ❖ fault tolerance
 - ❖ fault prevention
 - ❖ fault removal
 - ❖ fault forecasting

Fault tolerance

- 
- Aim: development of a system which functions correctly in presence of faults
 - achieved by some kind of **redundancy**
 - ❖ redundancy allows either to detect or to mask a fault
 - ❖ the goal is to reconfigure system to remove faulty components

Fault tolerance



- The redundancy allows fault detection/masking
- Redundancy is necessary, but not sufficient for fault tolerance
 - ❖ For example, two duplicated components connected in parallel do not make a system fault-tolerant, unless some form of **monitoring** is provided, which analyzes the results and selects the correct one.
 - ❖ fault location
 - ❖ fault containment
 - ❖ fault recovery

Fault masking



Fault masking is the process of insuring that only correct values get passed to the system output in spite of the presence of a fault

- ❑ This is done by preventing the system from being affected by errors by either correcting the error, or compensating for it in some fashion.
- ❑ For example:
 - ❖ a memory protected by an error-correcting code
 - ❖ triple modular redundancy with the majority voting

Fault detection



Fault detection is the process of recognizing
that a fault has occurred

❑ Example:

- ❖ acceptance tests in SW
- ❖ comparison in HW

Fault location



Fault location is the process of determining where a fault has occurred

- ❑ A failed acceptance test cannot generally be used to locate a fault. It can only tell that something has gone wrong.
- ❑ Similarly, when a disagreement occurs during the comparison of two modules, it is not possible to tell which of the two has failed

Fault containment



Fault containment is the process of isolating a fault and preventing its effect to propagate throughout a system

- ❑ This is achieved by
 - ❖ frequent fault detection
 - ❖ by multiple request/confirmation protocols
 - ❖ by performing consistency checks between modules

Reference

- 
- Slides:
 - ❖ E. Dubrova, Fault-Tolerant Design, Chapter 2
 - ❖ E. Dubrova, Design of Fault Tolerant Systems Course (Lecture 1 & 2)